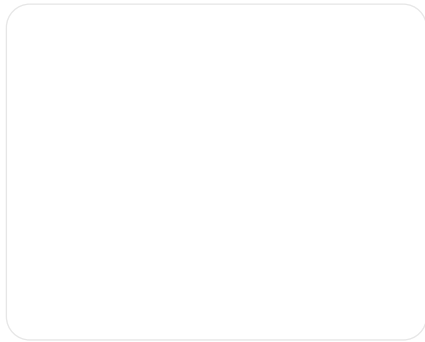


Excellent question.

This is **exactly the right time** to ask this — before you scale and before mistakes harden.

Below is a **complete, practical integration handbook** for ~10 modules using Bazel, written from **real-world failure patterns**, not theory.

I'll structure this into **three clear sections**, exactly as you asked.



Bazel Integration Handbook (10-Module Project)

Using Bazel

CONCEPTS YOU MUST USE (AND WHY)

(From Scratch → Production-Ready)

These are **NON-OPTIONAL** if you want a clean integration.

1. One Module = One Responsibility

What to do

- Each module = one logical responsibility
- Each module = its own directory + BUILD file

```
core/  
drivers/  
perception/  
planning/  
control/  
apps/
```

Why

- Smaller rebuilds
- Clear ownership
- Easier refactoring

If you skip this

- ➔ Huge rebuilds
- ➔ Cyclic deps
- ➔ Architecture rot

🔑 2. Use `cc_library` for Everything Except Final Apps

Rule

Code Type	Rule
Reusable code	<code>cc_library</code>
Executable	<code>cc_binary</code>
Tests	<code>cc_test</code>

Why

- Libraries scale
- Binaries do not

🔑 3. Explicit deps (STRICT DEPS)

Rule (MEMORIZE)

If you include a header, you must declare the dependency.

```
#include "math/math.h"
```

```
deps = ["//core/math:math"]
```

Why

- Prevents hidden dependencies
 - Safe refactoring
 - Accurate incremental builds
-

4. Public vs Private Headers (hdrs vs srcs)

Correct pattern

```
cc_library(  
    name = "math",  
    hdrs = ["math.h"],           # public API  
    srcs = ["math.cpp"],         # implementation  
)
```

Why

- Prevents header leakage
 - Enforces API boundaries
-

5. Visibility = Architecture Enforcement

Use visibility to encode layers

```
visibility = ["//perception:__subpackages__"]
```

Why

- Prevents wrong-direction dependencies
 - Stops “quick fixes” from becoming permanent
-

6. Centralized `config_setting + select()`

Best practice

```
build/configs/BUILD
```

Why

- One source of truth
 - Predictable CI behavior
 - No config duplication
-

7. Clean Include Style (NO RELATIVE INCLUDES)

Correct

```
#include "drivers/lidar/lidar.h"
```

Forbidden

```
#include "../lidar/lidar.h"
```

Why

- Hermetic builds
 - Safe directory refactors
-

8. External Deps Are Hermetic (WORKSPACE)

Rule

- Never rely on `/usr/include`
- Always use `http_archive` / `local_repository`

Why

- CI parity
 - Reproducible builds
 - No “works on my machine”
-

2 HOW TO AVOID COMMON MISTAKES (STEP-BY-STEP)

This is the **preventive checklist**.

● PHASE 1 — Before Writing Any BUILD File

- ✓ Define layers (core → drivers → app)
 - ✓ Decide public APIs
 - ✓ Decide which modules are reusable
 - ✓ Decide who is allowed to depend on whom
- ✗ If you skip this, Bazel errors will explode later
-

● PHASE 2 — While Writing BUILD Files

- ✓ Small targets
 - ✓ Explicit `srcs` (avoid `glob()` early)
 - ✓ Explicit `deps`
 - ✓ Minimal `visibility`
 - ✓ No `includes` unless third-party
-

● PHASE 3 — During Integration

- ✓ Fix errors by **architecture**, not hacks
 - ✓ Never “just add deps until it builds”
 - ✓ Never make everything public
 - ✓ Never include headers you don’t own
-

● PHASE 4 — Scaling to 10 Modules

- ✓ Centralize configs
 - ✓ Lock down visibility
 - ✓ Split large libraries
 - ✓ Introduce `common` only when necessary
-

3 WHAT MISTAKE CAUSES WHAT ERROR (CRITICAL MAP)

This section is **pure gold** — save it.

✗ MISTAKE → ! ERROR → ✓ FIX

✗ Include header but forgot dep

Error

```
undeclared inclusion(s) in rule
```

Fix

```
deps += ["//module:owner"]
```

✗ Header not in hdrs

Error

```
fatal error: xxx.h: No such file or directory
```

Fix

Move header to `hdrs`

✗ Using transitive dependency

Error

```
missing dependency declaration
```

Fix

Add direct dep (even if parent has it)

✗ Wrong visibility

Error

```
Target is not visible from target
```

Fix

Add correct `visibility` , NOT `public`

✗ Relative include paths

Error

```
file not found
```

Fix

Use package-based includes

✗ .cpp **not in** `srcs`

Error

```
cannot open source file
```

Fix

Add file to `srcs`

✗ Same source in two targets

Error

```
multiple definition of symbol
```

Fix

Move shared code to new library

✗ Cyclic deps

Error

`cycle in dependency graph`

Fix

Extract interfaces / common module

✗ Missing `strip_prefix` (external dep)

Error

`no BUILD file found in repository`

Fix

Set correct `strip_prefix`

✗ Missing default in `select()`

Error

`no matching condition`

Fix

Add `//conditions:default`

✗ IDE error but Bazel builds

Error

`cannot open source file (IDE only)`

Fix

Generate `compile_commands` / Bazel plugin

MASTER RULES (PRINT THIS)

- 1 BUILD files are **architecture**, not scripts
 - 2 If Bazel complains, **design is wrong**, not Bazel
 - 3 Headers = dependencies
 - 4 Visibility = layer boundaries
 - 5 Explicit beats implicit
 - 6 Small targets scale, big ones don't
-

RECOMMENDED NEXT STEPS (YOU CHOOSE)

Given you're integrating **10 modules**, best next topics are:

★ **Option A (Most Practical)**

👉 **Lesson 5 — External Dependencies (WORKSPACE)**

★ **Option B (Scaling Safely)**

👉 **Bazel Query** (`bazel query` , `cquery`)

★ **Option C (Automotive / Embedded)**

👉 **Toolchains & Cross-Compilation**

★ **Option D (Quality)**

👉 **Testing & CI with Bazel**

Reply with **one option**, and I'll go **deep and hands-on**.